Composition-based forestgap redesign, method coverage, and notebook migration

Summary

Release `forestgap` 1.0.0 as a composition-based package. Replace dynamic wrapper subclasses with `Proximity`, split construction/adaptation/application responsibilities into dedicated package-modules, and migrate every experiment, test, notebook, and document away from `with_sparse_proximity` and `forestgap.core`.

The implementation must provide an explicit proximity utility or an explicit unsupported status for every non-reference method in `application_roadmap.md`; it must never apply RF-GAP semantics where the required bootstrap/honesty contract is absent.

Package structure and public API

- Create three top-level package-modules:
  - `forestgap.proximity`: `Proximity`, sparse construction, relation dispatch, extension, prediction-weight reconstruction, affinity/distance utilities, and diagnostics.
  - `forestgap.adapters`: estimator capability registry plus sklearn, aeon, treeple, CCF, quantile, survival, and synthetic-null adapters.
  - `forestgap.applications`: standalone proximity consumers, split into `imputation`, `anomaly`, `importance`, `prototypes`, `visualization`, `prediction`, `clustering`, `quantiles`, `conformal`, `active_learning`, `attribution`, `retrieval`, and `feature_selection`.
- Remove `core.py`, `SparseProximityMixin`, and `with_sparse_proximity`; do not retain a compatibility shim.
- Define the public composition API:

```
forest = RandomForestClassifier(n_estimators=300, random_state=0)
proximity = Proximity(forest, relation="rfgap").fit(X_train, y_train)

P_train = proximity.get_proximities()
P_test = proximity.prox_extend(X_test)
scores = outlier_scores(proximity)
```

- Support both lifecycle paths: `.fit(X, y)` fits then indexes the supplied estimator; `.attach(X, y)` indexes an already fitted estimator. Expose the forest as `proximity.estimator`.
- Preserve the existing proximity operation names on `Proximity`: `get_proximities`, `prox_extend`, and `prox_predict`. Add `get_relation`, `extend_relation`, `diagnostics`, `capabilities_`, and `relation_metadata_`.

- Publish a migration guide mapping every removed wrapper/bound method to its `Proximity` or standalone application equivalent.

Methods, relations, and scientific validity

- Build a capability registry that declares leaf IDs, tree inventory, paths, bootstrap/in-bag/OOB state, honest roles, contribution weights, optional dependency, task type, and valid relations. Invalid requests raise `UnsupportedRelationError`; runners write an `unsupported` result row.
- Retain `original`, `oob`, `rfgap`, and `honest_gap`; add:
  - `kerf`: symmetric leaf-mass-normalized kernel with PSD tests;
  - `path`: normalized shared decision-path kernel with PSD tests;
  - `random_height`: seeded, cached per-tree height-cut partition kernel.
- Support methods from the roadmap as follows:
  - RF, ExtraTrees, RotationForest, and treeple oblique forests: original; OOB/RF-GAP only after each adapter proves bootstrap/OOB/contribution capability.
  - HonestForest: `honest_gap` only for prediction identity.
  - Isolation/extended isolation: native `score_samples` baseline only; optionally expose a separately named collision relation, never OOB/RF-GAP.
  - CCF: native packaged sklearn-compatible classifier/regressor, validated against deterministic MATLAB oracle fixtures before enabling any relation beyond original. Reference implementation
  - Quantile forest: native quantile prediction and conditional-distribution weights, not RF-GAP. API
  - Random survival forest: native risk/survival/hazard outputs plus a separately named survival-leaf relation and censoring-safe evaluation. RSF guide
  - Shi–Horvath URF: observed-versus-synthetic-null meta-estimator with persisted synthetic generator/seed and observed-row original/OOB relations.
  - Boosted, causal, Bayesian, PF-GAP/structured, multiview, graph, and autoencoding methods: visible registry rows with an explicit gated status until their separate relation semantics and leakage protocols are implemented.
- Retain treeple as an optional, non-vendored dependency. treeple documentation

Applications, experiments, and tests

- Port current Tier 0 behaviors to standalone functions: imputation, native/proximity anomaly scoring, local feature importance, prototypes, retrieval primitives, prediction identity, and visualization.
- Build Tier 1 utilities: kernel/kNN prediction, clustering, conditional distributions/quantiles, split conformal prediction, batch active learning, instance attribution, retrieval, and proximity-neighborhood feature selection.
- Route visualization through shared affinity/distance conversion; repair MDS so implicit sparse zero affinity becomes maximal distance and archive prior MDS artifacts as legacy.
- Replace current tests with:
  - `tests/proximity/` for lifecycle, sparse construction, relations, extension, diagnostics, and manifold conversion;
  - `tests/adapters/` for every estimator capability contract;

    - `tests/applications/` for each standalone application;
    - `tests/integration/` for exports, optional dependencies, method cells, CLI runners, and notebooks.
- Rework runners around `estimator × relation × application × data regime`, reuse fitted parent models/splits, and record revision, versions, seeds, split hashes, relation diagnostics, parent/downstream metrics, runtime, memory, and status.

Notebook migration and new tutorials

- Update all 13 existing notebooks and `oblique_rf_simulations.md` to instantiate a forest, create `Proximity`, and call standalone application functions. Remove every wrapper import and bound application method.
- Correct notebook narratives as part of migration:
    - replace isolation RF-GAP/OOB claims with native isolation scores and, where retained, an explicitly distinct collision relation;
    - use `honest_gap` for HonestForest identity demonstrations;
    - state relation directionality, symmetrization, and prediction-identity limits before visualisation, clustering, or attribution claims.
- Migrate existing notebooks by use case:
    - prediction identity: `honest_gap_prediction_identity`;
    - anomaly: both isolation notebooks, `honest_forest_outlier_demo_labeled`, and imputation/outlier notebook;
    - application tutorials: imputation, local importance, prototypes, spectral clustering, and manifold notebooks;
    - estimator comparisons: oblique, rotation, and simulation notebooks.
- Add four focused, executable tutorials:
    - `canonical_correlation_forest.ipynb`: CCF fit, original relation, oracle-conformance summary, and gated OOB/RF-GAP status;
    - `quantile_forest_proximities.ipynb`: conditional weights, quantiles, calibration, and comparison with native quantile predictions;
    - `random_survival_forest_proximities.ipynb`: censored split, survival-leaf relation, survival outputs, and censoring-safe metrics;
    - `synthetic_null_urf.ipynb`: synthetic-null construction, original/OOB relation, anomaly/clustering use, and seed provenance.
- Add notebook parameters for deterministic smoke execution: synthetic/local data by default, fixed seeds, reduced estimator/sample counts, no OpenML/network dependency, and documented optional-extra skips.
- Execute the notebook smoke suite in a clean kernel with `nbclient`; refresh and commit outputs for every migrated/new notebook using the smoke configuration. Add CI checks that fail on stale wrapper imports, invalid relation claims, missing execution metadata, or notebook execution failures.

Assumptions

- Existing notebooks remain maintained demonstrations rather than archived legacy material.
- New methods receive individual tutorial notebooks.

- Refreshed, deterministic smoke outputs are committed with the notebooks.
- Full data-scale experiments remain runner-based; notebooks demonstrate valid APIs and methodology rather than serving as benchmark artifacts.